Document number: P3198R0
Date: 2024-03-29
Audience: SG21
Reply-to: *Andrzej Krzemieński <akrzemi1 at gmail dot com>*

A takeaway from the Tokyo LEWG meeting on Contracts MVP

This paper lists potential modifications to be applied to [P2900R6] (Contracts for C++) based on the feedback obtained from people present at the LEWG meeting in Tokyo on the Contracts MVP. This includes the use case in [P3191R0] (Feedback on the scalability of contract violations handlers in P2900).

1. The underlying type of enums is redundantly specified to as `int`

The enumeration declarations in the paper have the form

```
enum class contract_kind : int {
  pre = 1,
  post = 2,
  assert = 3
};
```

The underlying type of an enumeration when you do not specify it is `int` anyway. Specifying it explicitly is redundant and not harmonized with the rest of the Standard library description.

Proposal 1. Drop the explicit specification of the underlying type; mention in the front matter of P2900 that the design intent is that the underlying type needs to be large enough to hold all possible values (including any vendor-provided ones, which start at 1000).

2. Conflating the contract violation with the predicate throw detection

In [P2900R6] the handler is called in two cases:

1. when a contract is violated (the predicate returned `false`),
2. when the evaluation of the predicate throws.

The second case is nothing like the contract violation, yet the library names used during the handling of this situation still have "contract violation" in their name.

This interacts with [P3191R0] which is concerned with having to generate a `try-catch` block for every predicate in order to satisfy the requirements.

Proposal 2. Introduce a different nomenclature to P2900. Use term "contract verification failure" in place of "contract violation". Rename:

1. `contract_violation` --> `contract_verification_failure`,
2. `handle_contract_violation` --> `handle_contract_verification_failure`.

Proposal 3. Use separate handlers for the two situations.

3. Redundant word "contract" in qualified names

Library names reside in namespace `contracts` and all but one have prefix `contract`:

```
std::contracts::contract_violation
std::contracts::contract_kind
```

Proposal 4. Drop namespace `contracts`.

Proposal 5. Rename the entities to avoid prefix `contract`:

1. `contract_kind` --> `assertion_kind`,
2. `contract_semantic` --> `evaluation_semantic`.

4. Too verbose name `invoke_default_contract_violation_handler`

The name is too long, and prefix `invoke_` brings no value.

Proposal 6. Rename to `default_contract_violation_handler`.

Proposal 7. Rename to `default_handle_contract_violation`.

5. Provide rationale for adding a namespace

The Standard library avoids using nested namespaces. Departure from this requires a strong rationale.

Our present rationale is:

1. If these names will be used at all, they will be used in a single translation unit.
2. We intend to extend the set of entities related to violation handling in the future. We want to reserve the space for introducing names that would potentially clash with names in `std`.
3. When user does `import std;` we do not want the handler-related functions to be imported.

6. Add another evaluation mode ([P3191R0])

[P3191R0] argues that the present evaluation mode *enforce* causes unnecessary performance compromises due to the contract violation handler machinery.

It requests a fourth, performance-sensitive predicate evaluation semantic where:

- the predicate is runtime-evaluated,
- no violation handler is invoked,
- program is stopped in an implementation-defined way.

7. Detection of throwing is too expensive ([P3191R0])

[P3191R0] argues that the requirement on the implementation to detect a throw from the predicate implies a code bloat in terms of injected `try-catch` blocks.

Proposal 8. When a predicate throws, call `terminate()` directly.

Proposal 9. Require a predicate to be `noexcept`.

Proposal 10. Just propagate such exception normally.

Proposal 11. Implementation-defined between `terminate()` and a violation handler.

8. Don't force implementations to use `abort()` ([P3191R0])

Calling `std::abort()`:

- has some performance implications,
- requires importing a library symbol,
- performs actions, such as raising `SIGABRT` that might be undesired when handling a contract violation.

Proposal 12. Revert to terminating in an implementation-defined manner.

9. ODR-concerns about handler replacement ([P3191R0])

[P3191R0] observes that the mechanism required for installing a custom violation handler à la `operator new` has the potential to trigger a non-benign ODR violation. ([P3191R0] calls this item unactionable.)

Proposal 13. Preserve the interface of the handler, but say that it is implementation-defined how it is installed.

10. Do not use word `contract` for names in namespace `contracts`

Assuming that namespace `contracts` stays, do not use word `contract` for entities defined in that namespace. This is to avoid repetition.

The initial response from SG21 was that names in the namespace are not mechanically prefixed with "contract". Not all names in the namespace start with "contract", only those where omitting the name looses meaning.

Proposal 14. Rename `contract_violation` to `violation`.

Proposal 15. Rename `invoke_default_contract_violation_handler` to `invoke_default_violation_handler`.

References

- [P2900R6] — Joshua Berne, Timur Doumler, Andrzej Krzemieński, "Contracts for C++", ("https://isocpp.org/files/papers/P2900R6.pdf").
- [P3191R0] — Louis Dionne, Yeoul Na, Konstantin Varlamov, "Feedback on the scalability of contract violations handlers in P2900",